Module 4: Network Layer

Complete Question & Answer Notes

Q1. Shortest Path Routing Algorithm

The goal is to find the path between source and destination with the **minimum cost** (fewest hops, least delay, highest bandwidth, etc.).

1.1 Dijkstra's Algorithm

The most common shortest path algorithm. It builds the shortest path tree from a source node by greedily picking the nearest unvisited node.

Algorithm Steps

1. Initialize: distance to source = 0; all others = ∞. Mark all nodes unvisited.
2. Pick the unvisited node with the smallest known distance call it u.
3. For each neighbor v of u: if $dist[u] + weight(u, v) < dist[v]$, update $dist[v]$.
4. Mark u as visited.
5. Repeat steps 2–4 until all nodes visited.

Example: Nodes A, B, C, D. Edges: A–B(2), A–C(4), B–C(1), B–D(5), C–D(1).

From A: $dist[B] = 2$, $dist[C] = 3$ (via B), $dist[D] = 4$ (via C). Shortest path A→D = A→B→C→D (cost 4).

Complexity: $O(V^2)$ with simple arrays; $O((V + E) \log V)$ with a priority queue.

1.2 Bellman-Ford Algorithm

Handles **negative edge weights**. Relaxes all edges $V - 1$ times.

$$dist[v] = \min(dist[v],\ dist[u] + weight(u, v))$$

Complexity: $O(V E)$. Used as the basis for distance vector routing.

Q2. Flow-Based Routing

Flow-based routing considers the **traffic load** (flow) on each link, not just topology. The aim is to minimize average packet delay across the network.

Key idea: Each link has a capacity C_i (packets/sec). Traffic λ_i flows on it. Mean delay on link i by M/M/1 queuing model:

$$d_i = \frac{1}{C_i - \lambda_i}$$

Total network delay: $T = \frac{1}{\gamma} \sum_i \lambda_i \cdot d_i$, where γ = total input traffic.

Approach:

1. Given expected traffic matrix (who sends what to whom), compute λ_i for all links.
2. Use optimization (e.g., flow deviation method) to find routing fractions that minimize T.

3. Routes are adjusted to shift traffic away from congested links.

Limitation: Requires global knowledge of traffic; not practical for dynamic real-world routing mainly used in *traffic engineering* and network planning.

Q3. Distance Vector Routing

Each router maintains a **routing table** (distance vector) listing the best known distance to every destination and the next hop to use.

How it Works

1. Initially, each router knows only its directly connected neighbors and their costs.
2. Periodically, each router sends its entire distance vector to all neighbors.
3. On receiving a neighbor's vector, a router updates its table using:

$$D_x(y) = \min_v \{cost(x, v) + D_v(y)\} \quad \text{(Bellman-Ford equation)}$$

4. Repeat until no more updates occur (convergence).

Problems:

- **Count-to-infinity:** When a link goes down, routers may keep incrementing distance and routing in a loop.
- **Fixes:** Split horizon (don't advertise a route back to where you learned it); Poison reverse (advertise cost $= \infty$ back to sender).

Slow convergence is a known drawback it takes many update cycles after a topology change. Used in: **RIP** (Routing Information Protocol).

Q4. Broadcast and Multicast Routing

4.1 Broadcast Routing

A packet is delivered to **all nodes** in the network.

- **Flooding:** Every router retransmits the packet on every link except the one it arrived on. Simple but causes huge traffic. Sequence numbers prevent loops.
- **Reverse Path Forwarding (RPF):** A router forwards a broadcast packet only if it arrives on the link that is on the shortest path *back to the source.* Efficient and loop-free.
- **Spanning Tree Broadcast:** Build a spanning tree of the network; broadcast only along tree edges no duplicates.

4.2 Multicast Routing

A packet is delivered to a **specific group** of hosts (not all, not one).

- Hosts join a multicast group using **IGMP** (Internet Group Management Protocol).
- Routers build a **multicast tree** to reach all group members.
- Two tree types:
  - **Source-based tree:** Separate shortest-path tree per source (e.g., DVMRP, PIM-DM).
  - **Shared tree (Rendezvous Point):** All sources share one tree rooted at an RP (e.g., PIM-SM).

- Class D IP addresses (224.0.0.0 – 239.255.255.255) are reserved for multicast.

Q5. IPv4 Addressing with Classes

An IPv4 address is a **32-bit** number, usually written in dotted-decimal notation (e.g., 192.168.1.10). It has two parts: **Network ID** and **Host ID**.

Classful addressing divides the address space into fixed classes:

Class	1st Octet Range	Net Bits	Host Bits	Max Hosts
A	1 – 126	8	24	16,777,214
B	128 – 191	16	16	65,534
C	192 – 223	24	8	254
D	224 – 239	Multicast (no host field)		
E	240 – 255	Reserved / Experimental		

Special addresses:

- Host bits all 0 → Network address (e.g., 192.168.1.**0**)
- Host bits all 1 → Broadcast (e.g., 192.168.1.**255**)
- 127.0.0.1 → Loopback (localhost)
- 10.x.x.x, 172.16–31.x.x, 192.168.x.x → Private addresses (RFC 1918)

Limitation: Classful addressing wastes huge blocks of addresses (e.g., a company needing 300 hosts must get a Class B with 65534 addresses). This led to CIDR.

Q6. IPv6: Features and Structure

IPv4 is running out of addresses (only $2^{32} \approx 4$ billion). IPv6 was designed to replace it.

6.1 Key Features

- **128-bit addresses** 2^{128} possible addresses (effectively unlimited).
- **Simplified header** Fixed 40-byte header (fewer fields than IPv4).
- **No fragmentation by routers** only at source; routers drop oversized packets and send ICMPv6 "Packet Too Big."
- **No broadcast** replaced by multicast and anycast.
- **Built-in IPsec** security is native, not an add-on.
- **Stateless Address Autoconfiguration (SLAAC)** hosts can self-configure addresses.
- **Flow label field** for QoS and real-time traffic identification.

6.2 IPv6 Address Notation

Written as 8 groups of 4 hex digits separated by colons:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

Abbreviations: leading zeros in a group can be dropped; consecutive all-zero groups replaced by `::` (once only).

6.3 IPv6 Header Fields

Field	Purpose
Version (4b)	Always 6
Traffic Class (8b)	QoS / DSCP
Flow Label (20b)	Identify packet flows
Payload Length (16b)	Length of data after header
Next Header (8b)	Type of next header (like Protocol field in IPv4)
Hop Limit (8b)	Like TTL decremented each hop
Source Address (128b)	Sender
Destination Address (128b)	Receiver

Q7. Subnetting

Subnetting divides a large network into smaller **sub-networks (subnets)**, reducing broadcast domains and improving management.

A **subnet mask** tells which bits of the IP address are the network/subnet part and which are the host part. Written as a 32-bit mask (e.g., 255.255.255.0) or as prefix length (e.g., /24).

Example — Subnet a Class C network 192.168.10.0/24 into 4 subnets

We need 4 subnets ⇒ borrow 2 bits from host part ($2^2 = 4$).
New prefix: /26, Subnet mask: **255.255.255.192**
Hosts per subnet: $2^6 - 2 = 62$ usable hosts.

Subnet	Network Addr	Host Range	Broadcast
1	192.168.10.0	.1 – .62	192.168.10.63
2	192.168.10.64	.65 – .126	192.168.10.127
3	192.168.10.128	.129 – .190	192.168.10.191
4	192.168.10.192	.193 – .254	192.168.10.255

Key formulas:

$$\text{Number of subnets} = 2^s \quad (\text{s} = \text{subnet bits borrowed})$$

$$\text{Hosts per subnet} = 2^h - 2 \quad (\text{h} = \text{remaining host bits; subtract network \& broadcast})$$

Q8. CIDR (Classless Inter-Domain Routing)

CIDR was introduced in 1993 to solve the rapid exhaustion of IPv4 addresses caused by classful allocation. It removes the strict class boundaries.

In CIDR, an IP address is written as `address/prefix-length`, where the prefix length specifies exactly how many bits are the network part. This is called **slash notation** or **prefix notation**.

Benefits:

- Allocate blocks of *any* size, not just /8, /16, /24.

- **Route aggregation (supernetting):** Multiple contiguous networks can be advertised as a single prefix, reducing the size of routing tables.

Example

ISP has block `200.23.16.0/20`. It splits this among customers:

- Customer A: `200.23.16.0/23` (512 addresses)
- Customer B: `200.23.18.0/23` (512 addresses)
- Customer C: `200.23.20.0/22` (1024 addresses)

To the outside world, the ISP advertises only `200.23.16.0/20` one route instead of many. This is **route aggregation**.

Another example: `192.168.4.0/22` covers addresses 192.168.4.0 through 192.168.7.255 $\Rightarrow 2^{32-22} = 2^{10} = 1024$ addresses total.

Q9. Routing Protocols: RIP, OSPF, BGP

9.1 RIP (Routing Information Protocol)

- Based on **distance vector** routing; uses Bellman-Ford.
- Metric: **hop count** (max 15; 16 = unreachable).
- Sends full routing table to neighbors every **30 seconds**.
- Simple and easy to configure; suitable for **small networks**.
- Slow convergence; count-to-infinity problem.
- RIPv2 adds subnet mask support (CIDR); RIPng for IPv6.

9.2 OSPF (Open Shortest Path First)

- Based on **Link State** routing; uses Dijkstra's algorithm.
- Each router floods **Link State Advertisements (LSAs)** to build a complete topology map.
- Metric: configurable **cost** (based on bandwidth by default).
- Supports CIDR, VLSM, authentication, and multiple areas (hierarchical).
- **Area 0** = backbone; inter-area routing through Area Border Routers (ABRs).
- Fast convergence; scales to large networks. Interior Gateway Protocol (IGP).

9.3 BGP (Border Gateway Protocol)

- The **routing protocol of the Internet** used between Autonomous Systems (AS).
- Path vector protocol: advertises full AS path to prevent loops.
- Metric: **policy-based** (AS path length, local preference, MED, etc.) not just shortest path.
- Runs over **TCP port 179**; highly stable and scalable.
- iBGP: within an AS. eBGP: between different ASes.

Feature	RIP	OSPF	BGP
Type	Distance Vector	Link State	Path Vector
Algorithm	Bellman-Ford	Dijkstra	–
Metric	Hop count	Cost	Policy
Scope	Small LAN	Enterprise	Internet (inter-AS)
Convergence	Slow	Fast	Slow (stable)

Q10. NAT (Network Address Translation)

NAT allows multiple devices with **private IP addresses** to share a single **public IP address** when accessing the Internet.

> **Why NAT?** Private addresses (10.x.x.x, 172.16-31.x.x, 192.168.x.x) are not routable on the Internet. NAT translates them at the border router.

How it works: The NAT router maintains a **NAT translation table** mapping (private IP, port) $\leftrightarrow$ (public IP, port).

10.1 Types of NAT

1. Static NAT

- One-to-one permanent mapping: one private IP $\leftrightarrow$ one public IP.
- Used for servers that must be reachable from outside.

2. Dynamic NAT

- Maps private IPs to a *pool* of public IPs dynamically.
- First-come-first-served; if pool exhausted, new connections fail.

3. PAT / NAT Overload (Port Address Translation)

- Many private IPs share a *single* public IP, distinguished by **port numbers**.
- Most common type used in home routers.
- Also called *masquerading*.

Advantages: Conserves IPv4 addresses; adds security (internal IPs hidden).
Disadvantages: Breaks end-to-end connectivity; complicates some protocols (FTP, VoIP, IPsec); adds processing overhead.

Q11. ICMP (Internet Control Message Protocol)

ICMP operates at the **Network Layer** (alongside IP) and is used by routers and hosts to exchange **error messages and control information** about IP packet delivery. It does not carry user data.

> ICMP is defined in RFC 792. Every ICMP message is encapsulated in an IP packet with Protocol = 1.

11.1 ICMP Message Format

All ICMP messages share a common header:

Type (8 bits)	**Code** (8 bits)	**Checksum** (16 bits)
Message Body (varies by type)		

11.2 Key ICMP Messages and Functions

Type	Name	Direction	Purpose
0	Echo Reply	Router→Host	Response to ping
3	Destination Unreachable	Router→Host	Packet cannot be delivered (host/port/net unreachable)
5	Redirect	Router→Host	Tell host to use a better route
8	Echo Request	Host→Router	Ping test reachability
11	Time Exceeded	Router→Host	TTL reached 0 (used by `traceroute`)
12	Parameter Problem	Router→Host	Bad IP header

11.3 Key Uses

- **ping:** Sends ICMP Echo Request (Type 8); expects Echo Reply (Type 0). Tests connectivity and measures round-trip time.
- **traceroute:** Sends packets with TTL=1,2,3... Each router that drops the packet (TTL=0) sends back Time Exceeded (Type 11), revealing the path hop-by-hop.
- **Error reporting:** Routers notify sources when packets cannot be delivered.
- **Path MTU Discovery:** Source sends large packet; if router returns "Fragmentation Needed" (Type 3, Code 4), source reduces packet size.

End of Module 4 Notes — Network Layer